

MEF4

Matthew Abadir

2026-05-19

Assignment 4 - Template CRR and BS Models, Option Pricing and Implied Volatility Mathematical and Empirical Finance, 2025-2026 Author: Matthew Abadir, Finn van Eijk, Liza Engels Group: Liza Group 12, Matthew and Finn Group 42

R Markdown

```
rm(list = ls())

# -----
# BASELINE PARAMETERS (used in Questions 1-5)
# -----

# Set the baseline parameters here
S0    <- 100
K      <- 100
r      <- 0.02
sigma  <- 0.30
delta  <- 0.01
TT     <- 1
```

QUESTION 1 - CRR European option prices

```
EuroCRR <- function(N) {

  # Define here the function returning the price of the specified option
  dt    <- TT / N
  u      <- exp(sigma * sqrt(dt))
  d      <- 1 / u
  q      <- (exp((r - delta) * dt) - d) / (u - d)
  j      <- 0:N
  ST     <- S0 * u^(N - j) * d^j
  V      <- pmax(ST - K, 0)
  disc   <- exp(-r * dt)
  for (n in N:1) {
    V <- disc * (q * V[1:n] + (1 - q) * V[2:(n + 1)])
  }
  return(V[1])
}
```

```

# Compute CRR prices on the grid of N values
N <- c(5, 10, 20, 50, 100, 250, 500, 1000)
CRR <- numeric(length(N)) # preallocate space
for (i in 1:length(N)) {
  CRR[i] <- EuroCRR(N[i]) # fill CRR
}

# Display table of results
data.frame(N = N, CRR_Price = round(CRR, 4))

```

```

##      N CRR_Price
## 1     5   12.8333
## 2    10   11.9555
## 3    20   12.0993
## 4    50   12.1866
## 5   100   12.2159
## 6   250   12.2335
## 7   500   12.2393
## 8  1000   12.2423

```

QUESTION 2 - Black-Scholes pricing function

```

BSPrice <- function(TT, S0, K, r, sigma, delta, type) {

  # Define here the function returning the BS price
  d1 <- (log(S0 / K) + (r - delta + 0.5 * sigma^2) * TT) / (sigma * sqrt(TT))
  d2 <- d1 - sigma * sqrt(TT)
  if (type == "call") {
    price <- S0 * exp(-delta * TT) * pnorm(d1) - K * exp(-r * TT) * pnorm(d2)
  } else {
    price <- K * exp(-r * TT) * pnorm(-d2) - S0 * exp(-delta * TT) * pnorm(-d1)
  }
  return(price)
}

# BS price of European option with baseline parameters
BS <- BSPrice(TT, S0, K, r, sigma, delta, "call")
BS

```

```
## [1] 12.2452
```

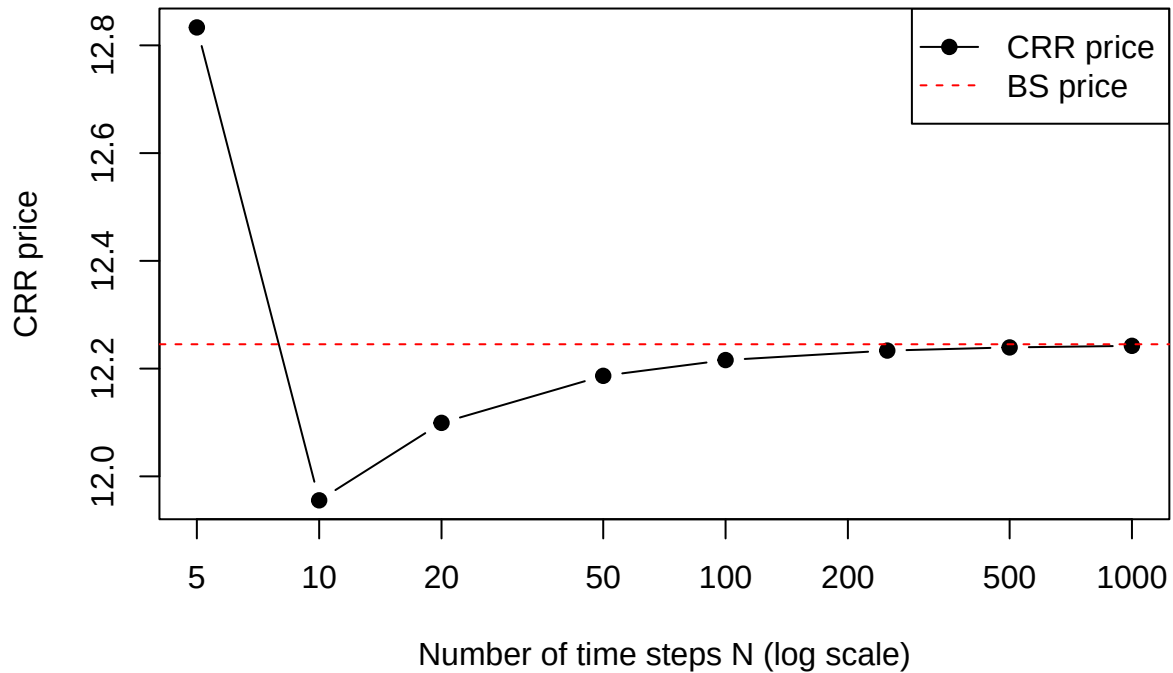
QUESTION 3 - Plot CRR

```

plot(x = N,
     y = CRR,
     type = "b",
     log = "x",
     pch = 19,
     xlab = "Number of time steps N (log scale)",
     ylab = "CRR price")
abline(h = BS, col = "red", lty = 2)

```

```
legend("topright", legend = c("CRR price", "BS price"),
      col = c("black", "red"), lty = c(1, 2), pch = c(19, NA))
```



QUESTION 4 - Digital CRR option

```
# Set the tree parameters
N <- 100
dt <- TT / N
u <- exp(sigma * sqrt(dt))
d <- 1 / u
q <- (exp((r - delta) * dt) - d) / (u - d)

# Create here the stock tree (same as before)
S <- matrix(NA, N + 1, N + 1)
S[1, 1] <- S0
for (n in 2:(N + 1)) {
  for (j in 1:n) {
    S[j, n] <- S0 * u^(n - j) * d^(j - 1)
  }
}

# Digital payoff at maturity
V <- matrix(NA, N + 1, N + 1)
V[, N + 1] <- ifelse(S[, N + 1] > K, 1, 0)

# Backward induction
```

```

disc <- exp(-r * dt)
for (n in N:1) {
  for (j in 1:n) {
    V[j, n] <- disc * (q * V[j, n + 1] + (1 - q) * V[j + 1, n + 1])
  }
}

round(V[1, 1], 4)

```

```
## [1] 0.4834
```

QUESTION 5 - Path dependent option

```

# CAREFUL: this is a path dependent option, the backward induction doesn't work!

# Set the tree parameters
N <- 12
dt <- TT / N
u <- exp(sigma * sqrt(dt))
d <- 1 / u
q <- (exp((r - delta) * dt) - d) / (u - d)

# All 2^N up/down sequences
Paths <- expand.grid(rep(list(c(u, d)), N))

# Preallocate output vectors
Payoffs <- numeric(2^N)
PathProb <- numeric(2^N)

# Walk each path
for (i in 1:(2^N)) {

  S_path <- numeric(N)      # Preallocate values of stock path starting from S0

  s <- S0
  moves <- as.numeric(Paths[i, ])
  for (k in 1:N) {
    s <- s * moves[k]
    S_path[k] <- s
  } # Fill price path

  S_ast <- mean(S_path)      # define S* (S_0 excluded)
  Payoffs[i] <- max(S_ast - K, 0)

  n_up <- sum(moves == u)    # total up-moves in path i
  PathProb[i] <- q^n_up * (1 - q)^(N - n_up) # probability of the path i
}

# Option price = discounted risk-neutral expected payoff
PathDepPrice <- exp(-r * TT) * sum(PathProb * Payoffs)
round(PathDepPrice, 4)

```

```
## [1] 7.5622
```

```
# Compare with European option (same N = 12 steps)
EuroPrice12 <- EuroCRR(12)
round(EuroPrice12,4)
```

```
## [1] 12.0032
```

QUESTION 6 - Implied Volatility

```
OptionData <- read.csv("C:/Users/Matth/Documents/Assignment_4_dataset.csv")
n_opt      <- nrow(OptionData)      # number of options

ImpliedVol <- function(price, TT, S0, K, r, delta, type) {

  # Define here the function
  f <- function(s) BSPrice(TT, S0, K, r, s, delta, type) - price
  lo <- 1e-6
  hi <- 5
  if (f(lo) * f(hi) > 0) return(NA_real_)
  uniroot(f, lower = lo, upper = hi, tol = 1e-10)$root
}

# Loop over the options
IVs <- numeric(n_opt)
for (i in 1:n_opt) {
  IVs[i] <- ImpliedVol(
    price = OptionData$MidPrice[i],
    TT    = OptionData$Maturity[i],
    S0    = OptionData$StockPrice[i],
    K     = OptionData$Strike[i],
    r     = OptionData$RiskFree[i],
    delta = OptionData$DividendYield[i],
    type  = tolower(OptionData$Type[i])
  )
}

OptionData$IV <- IVs

# Table of the first six options
ResultTable <- data.frame(
  Strike   = OptionData$Strike,
  Type     = OptionData$Type,
  MidPrice = OptionData$MidPrice,
  IV_pct   = round(100 * OptionData$IV, 2)
)
head(ResultTable)
```

```
##   Strike Type MidPrice IV_pct
## 1   250 Put    0.220  82.03
## 2   265 Put    0.275  76.07
## 3   630 Call   0.450  63.49
## 4   660 Call   0.315  66.66
## 5   670 Call   0.285  67.77
## 6   690 Call   0.225  69.57
```

```

# Plot IV (in %) vs strike (sorted in increasing strike order)
OptionData <- OptionData[order(OptionData$Strike), ]      # sort the dataset
plot(x = OptionData$Strike ,
     y = 100 * OptionData$IV ,
     type = "b",
     pch = 19,
     xlab = "Strike",
     ylab = "Implied volatility (%)",
     main = "Implied Volatility curve")

```

